QUESTIONS 10 / 10 completed

Suffix Array

SUFFIX ARRAY

Write a C program to construct the Suffix Array of a given string. The program should generate all suffixes of the string, sort them lexicographically, and display their starting indices in sorted order.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int a[100], n, i, j, t;

    scanf("%99s", s);
    n = strlen(s);

    for(i=0;i<n;i++) a[i]=i;

    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(strcmp(s+a[i],s+a[j])>0) {
                t=a[i]; a[i]=a[j]; a[j]=t;
            }

    printf("Suffix Array: ");
    for(i=0;i<n;i++) printf("%d ",a[i]);

```

INPUT

banana

QUESTIONS 10 / 10 completed

Lexicographical Or

LEXICOGRAPHICAL ORDER

Develop a C program to generate all suffixes of a given string and display them in lexicographical order along with their corresponding indices obtained through the Suffix Array.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int a[100], n, i, j, t;

    scanf("%99s", s);
    n = strlen(s);

    for(i=0;i<n;i++) a[i]=i;

    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(strcmp(s+a[i],s+a[j])>0) {
                t=a[i]; a[i]=a[j]; a[j]=t;
            }

    for(i=0;i<n;i++)
        printf("%d %s\n", a[i], s+a[i]);

```

INPUT

banana

QUESTIONS 10 / 10 completed

Longest Common I

LONGEST COMMON PREFIX

Write a program to construct the Longest Common Prefix (LCP) Array for a given string after generating its Suffix Array. Display the LCP value between every pair of adjacent suffixes.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int a[100], n, i, j, k, t, lcp;

    scanf("%99s", s);
    n = strlen(s);

    for(i=0;i<n;i++) a[i]=i;

    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(strcmp(s+a[i],s+a[j])>0) {
                t=a[i]; a[i]=a[j]; a[j]=t;
            }

    printf("Suffix Array: ");
    for(i=0;i<n;i++) printf("%d ",a[i]);

```

INPUT

banana

QUESTIONS 10 / 10 completed

Matching Positions

MATCHING POSITIONS

Implement a pattern matching program using a Suffix Array. The program should search for a given pattern in the text efficiently and report all matching positions.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[100], p[50];
    int sa[100], n, i, j, temp;

    scanf("%s %s", s, p);
    n = strlen(s);

    for (i = 0; i < n; i++) sa[i] = i;

    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (strcmp(s + sa[i], s + sa[j]) > 0) {
                temp = sa[i];
                sa[i] = sa[j];
                sa[j] = temp;
            }

    for (i = 0; i < n; i++)
```

INPUT

banana
ana

QUESTIONS 10 / 10 completed

LCP Array

LCP ARRAY

Write a C program to determine the longest repeated substring in a given string using the LCP Array. The program should display both the substring and its length.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int sa[100], lcp[100], n, i, j, temp;
    int max = 0, pos = 0;

    scanf("%99s", s);
    n = strlen(s);

    for (i = 0; i < n; i++) sa[i] = i;

    // Suffix Array
    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (strcmp(s + sa[i], s + sa[j]) > 0) {
                temp = sa[i]; sa[i] = sa[j]; sa[j] = temp;
            }

    // LCP Array
```

INPUT

banana

QUESTIONS 10 / 10 completed

Suffix & LCP Array

SUFFIX & LCP ARRAY

Develop a program that computes the total number of distinct substrings present in a given string using the Suffix Array and LCP Array concepts.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int sa[100], lcp[100], n, i, j, k, temp;
    int count = 0;

    scanf("%99s", s);
    n = strlen(s);

    for (i = 0; i < n; i++)
        sa[i] = i;

    // Suffix Array
    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (strcmp(s + sa[i], s + sa[j]) > 0) {
                temp = sa[i];
                sa[i] = sa[j];
                sa[j] = temp;
```

INPUT

banana

QUESTIONS 10 / 10 completed

LCP-2

LCP-2

Write a program to find the longest common substring between two input strings by constructing suffixes and applying Longest Common Prefix computations.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <string.h>

int main() {
    char a[100], b[100];
    int i, j, k, max = 0, pos = 0;

    scanf("%s %s", a, b);

    for (i = 0; a[i]; i++) {
        for (j = 0; b[j]; j++) {
            k = 0;
            while (a[i+k] && b[j+k] && a[i+k] == b[j+k])
                k++;

            if (k > max) {
                max = k;
                pos = i;
            }
        }
    }
```

INPUT

banana ananas

QUESTIONS 10 / 10 completed

Pattern Matching T

PATTERN MATCHING TECHNIQUES.

Implement both naive pattern matching and Suffix Array based pattern matching techniques. Compare their performance for multiple search operations on the same text.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX 1000

// Naive Pattern Matching
void naiveSearch(char text[], char pattern[]) {
    int n = strlen(text);
    int m = strlen(pattern);

    printf("Naive Search for \"%s\": ", pattern);

    for (int i = 0; i <= n - m; i++) {
        int j;
        for (j = 0; j < m; j++) {
            if (text[i + j] != pattern[j])
                break;
        }
        if (j == m)
```

INPUT

banana banana
bandana banana

QUESTIONS 10 / 10 completed

Kasai's Algorithm

KASAI'S ALGORITHM

Develop a C program to implement Kasai's Algorithm for efficient construction of the LCP Array from a previously generated Suffix Array. Display both arrays and analyze the time complexity.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <string.h>

void kasai(char *txt, int sa[], int lcp[], int n) {
    int rank[n], k = 0;

    for (int i = 0; i < n; i++)
        rank[sa[i]] = i;

    for (int i = 0; i < n; i++) {
        if (rank[i] == n - 1) {
            k = 0;
            continue;
        }

        int j = sa[rank[i] + 1];

        while (i + k < n && j + k < n && txt[i + k] == txt[j + k])
            k++;

        lcp[rank[i]] = k;
```

INPUT

String: banana
Suffix Array: 5 3 1 0 4 2